Trabajo Práctico I - Optimización de consultas

a. Resolver las siguientes consultas utilizando Explain Analyze y describir los planes de ejecución.

1. Devolver todos los comercios que se encuentran en la calle Mitre
2. Crear un índice hash por calle para los comercios y ejecutar la consulta anterior y comparar los resultados y el plan de ejecución
3. Devolver la cantidad de comercios agrupados por producto principal
4. Crear un índice B-Tree por producto principal y ejecutar la consulta anterior y comparar los resultados y el plan de ejecución
5. Crear un índice Hash por producto principal y ejecutar la consulta anterior y comparar los resultados y el plan de ejecución
6. Devolver todos los comercios que se encuentran en el barrio 1
7. Crear un índice hash para mejorar la consulta anterior y comparar los resultados y el plan de ejecución
8. Devolver todos los comercios que se encuentran en el barrio Centro, en calle galarza, con clanae correpondiente al rubro Kiosko
9. Devolver los pares de comercios del mismo rubro (clanae) que están cerca uno del otro (misma calle y número +-50)
10. Devolver los comercios que se encuentran en la calle 8 de junio con clanae 47119
11. Crear un btree para comercios, con calle y clanae. Volver a ejecutar la consulta anterior y verificar si cambió el plan
12. Escribir 3 consultas que contengan joins, subconsultas y/o agregaciones, verificar el plan de ejecución. Tratar de mejorar su eficiencia planteando la consulta de distinta manera y/o creando índices.

b. Dadas las siguientes tablas:

```
create table articulos as
        select id::int8, random_string(100) as descripcion, round(random() * 30)::int8 as rubro
        from generate_series(1, 500000) as id;

create table precios as
        select id::int8,
            round(random() * 99000)::int8+1 as id_articulo,
            timestamp '2022-01-01 00:00:00' + (random() * (interval '1000 days')) as fecha,
            100.00+round(random() * 5000)::numeric(11,2) as precio
        from generate_series(1, 8000000) as id;
```

- la funcion random_string está dada por:

```
CREATE OR REPLACE FUNCTION public.random_string(
        length integer)    RETURNS text    LANGUAGE 'plpgsql'
AS $BODY$
  declare
   chars text[] :=
'{0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S,T,U,V,W,X,Y,Z,a,b,c,d,e,f,g,h,i,j,k,l
,m,n,o,p,q,r,s,t,u,v,w,x,y,z}';
  result text := '';
  i integer := 0;
begin
  if length < 0 then
    raise exception 'Given length cannot be less than 0';
  end if;
  for i in 1..length loop
    result := result || chars[1+random()*(array_length(chars, 1)-1)];
  end loop;
  return result;
end;
$BODY$;
```

resuelva las consultas que se enuncian a continuación, explique su plan de ejecución e intente hacerlas más eficientes:

1. Artículos correspondientes al rubro 5
2. Precio actual del articulo 10
3. Precio actual y descripción del articulo 10
4. Artículos cuyos precios están entre 1000 y 1500
5. Precio promedio por articulo
6. Precio promedio por rubro
7. Máximo precio del rubro 15, posterior marzo de 2023
8. Maximo precio promedio por rubro
9. Máximo precio promedio por artículo

c. Crear una tabla con una gran cantidad de registros (más de 500,000 filas) que incluya al menos cuatro columnas con valores repetidos (por ejemplo, una columna de nombres, apellido, edad y ciudad)

1. Ejecutar “EXPLAIN ANALYZE SELECT * FROM personas WHERE ciudad = 'Colón';” y analizar y describir el plan de ejecución.

2. Crear un índice btree sobre la columna ciudad y volver a ejecutar y describir los resultados.
3. Eliminar el índice btree y crear un índice hash sobre la columna ciudad y volver a ejecutar y describir los resultados.
4. Realizar una consulta que busque las personas que estén dentro de un rango de edad, ejecutar EXPLAIN ANALYZE y describir los resultados
5. Crear un índice btree sobre la edad y volver a ejecutar y analizar.
6. Crear una consulta con funciones agregadas (count, sum, avg) y analizar el plan de ejecución con y sin índices sobre la columna de cálculo.

d. Dadas las siguientes relaciones

```
CREATE TABLE clientes (
  id SERIAL PRIMARY KEY,
  nombre TEXT,
  email TEXT UNIQUE
);

CREATE TABLE pedidos (
  id SERIAL PRIMARY KEY,
  cliente_id INT REFERENCES clientes(id),
  fecha DATE,
  total NUMERIC
);
```

1. Insertar 500k clientes y 5M pedidos distribuidos aleatoriamente
2. Ejecutar una consulta para obtener los 10 clientes con mayor facturación en 2024

```
SELECT c.nombre, SUM(p.total) AS facturacion
FROM clientes c
JOIN pedidos p ON c.id = p.cliente_id
WHERE p.fecha BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY c.id
ORDER BY facturacion DESC
LIMIT 10;
```

3. Explicar el plan de ejecución
4. Probar índices con distintas claves en Pedidos:
    - INDEX(cliente_id)

- INDEX(cliente_id, fecha)
- INDEX(fecha, cliente_id)

5. Analizar cuál da mejor rendimiento y justificar.